A length-nine gap in the fertility spectra of consecutive-pattern stack-sorting maps

Aurantium

14 October 2026

Abstract

For each of the single forbidden patterns 213 and 231, the fertility spectrum of the consecutive-pattern-avoiding stack-sorting map on $\mathfrak{S}_9$ is $\{0, 128\} \cup (\{1, \ldots, 64\} \setminus \{55\})$. An archived complete enumeration visits all $9! = 362\,880$ inputs for SC_{213} and records the full histogram, including 265 602 outputs with zero fertility. A step-by-step complement conjugacy gives the same histogram for SC_{231}. The missing value $55 \leq 2^{9-3}$ contradicts the fixed-length inclusion in Kemeklis's Conjecture 4.1. It does not contradict the theorem that every positive integer is a fertility when the length may vary. We specify the algorithm, its correspondence with the original definition, and the exact scope of the recorded execution.

1 The fixed-length statement

Write $\mathfrak{S}_n$ for the permutations of $[n] = \{1, \ldots, n\}$. Fix a *single* pattern $\sigma \in \mathfrak{S}_3$. The map SC_σ reads its input from left to right and maintains a stack read from *top to bottom*. With next input x, test the consecutive top triple that would result from pushing x. If its relative order is σ, pop the current top to the output and retry the same x. Otherwise push x and advance the input. After the input is consumed, pop the remaining stack to the output in top-to-bottom order. This is the rule in [1, p. 1].

For an output $\pi \in \mathfrak{S}_n$, define

$$f_{\sigma,n}(\pi) = \#\{\tau \in \mathfrak{S}_n : \mathrm{SC}_\sigma(\tau) = \pi\}, \qquad A_n^\sigma = \{f_{\sigma,n}(\pi) : \pi \in \mathfrak{S}_n\}.$$

Zero is included when some output is not attained.

Kemeklis's Conjecture 4.1 [1, p. 11] proposes the inclusion

$$\{1, 2, \ldots, 2^{n-3}\} \subseteq A_n^\sigma, \qquad \sigma \in \{213, 231\}. \tag{1}$$

For integer $n \geq 3$, this asks that every integer in the displayed range occur at that same length. The printed source uses the notation $\sigma = \{213, 231\}$ and the symbol $\subset$. In the context of the single-pattern maps defined there, we read this as the two separate choices $\sigma = 213$ and $\sigma = 231$, not a machine simultaneously avoiding both patterns. Our counterexample uses $n = 9$; it violates even the weak inclusion in (1), independently of whether the source's subset symbol is intended to require strict inclusion.

Theorem 1 (Recorded exhaustive computation and symmetry)**.** *For the two single-pattern maps,*

$$A_9^{213} = A_9^{231} = \{0, 128\} \cup (\{1, \ldots, 64\} \setminus \{55\}). \tag{2}$$

In particular, no length-nine output of either map has fertility 55*, so* (1) *fails at* $n = 9$*.*

The theorem has a finite computational component: it depends on the specified program and its completed execution. The symmetry component is proved below. This note does not claim a hand classification of all fibers or proof-assistant certification.

2 Why the program implements the defined map

For three distinct numbers a, b, c, standardization replaces each entry by one plus the number of smaller entries. For a prospective triple (x, s_0, s_1), with s_0 the stack top and s_1 immediately below it, the order is 213 exactly when

$$s_0 < x < s_1. \tag{3}$$

The recorded independent implementation computes all three ranks directly, rather than substituting a specialized inequality test. Its complete stack function is:

```
def sc(p):
    stack=deque(); output=[]
    for x in p:
        while len(stack)>=2:
            triple=(x,stack[0],stack[1])
            order=tuple(1+sum(b<a for b in triple)
                        for a in triple)
            if order!=(2,1,3): break
            output.append(stack.popleft())
        stack.appendleft(x)
    return tuple(output)+tuple(stack)
```

Here the left end of the deque is the top. The line wrapping of the rank expression is only typographical; the distributed program preserves the original mathematical function and enumeration logic.

Lemma 1. *For every $p \in \mathfrak{S}_n$, this function terminates and returns $\mathrm{SC}_{213}(p)$, with each input label appearing once in the output.*

Proof. Initially the stack is empty and avoids the forbidden consecutive pattern. Popping its top leaves a suffix of the top-to-bottom sequence, so cannot create a new consecutive triple. A push creates only one new triple, namely (x, s_0, s_1), when two entries were already in the stack. Thus testing only that triple suffices to maintain the invariant. If the stack has fewer than two entries, the push cannot create a length-three pattern.

The inner loop pops precisely when the prospective triple standardizes to $(2, 1, 3)$. A pop does not change the outer-loop input x, so the same input is retried against the updated stack. Breaking the loop, or reducing the stack below length two, is followed by pushing x at the top. This matches each decision in the defining rule.

For any fixed input entry, each unsuccessful trial removes a stack entry, so the inner loop terminates. Every label is pushed once and popped once, either during a trial or in the final drain. No label is created, duplicated or discarded. The final deque is already listed top-to-bottom, hence concatenating its tuple to the output is exactly the final drain. The output is therefore a permutation of $[n]$, and every step agrees with the original deterministic map. □

3 Enumeration, accounting and the complete histogram

The original computation iterates over `permutations(range(1,10))`. The permutation iterator supplies each ordering of the nine distinct labels exactly once, hence every element of $\mathfrak{S}_9$. The loop increments an input counter and, for each input p, increments `counts[sc(p)]` by one. Keys are complete ordered output tuples, not abbreviated permutation ranks or hash values. The counter uses exact Python integers; dictionary hash collisions are resolved by tuple equality and do not merge distinct outputs.

By Lemma 1, each loop iteration contributes once to the fiber of its unique output. Thus at the end, for every attained π, `counts[pi]` is exactly $f_{213,9}(\pi)$. There is no restriction to inputs whose outputs are sorted. The original program asserts that its input counter equals 9!.

Define the full histogram

$$h_9(f) = \#\{\pi \in \mathfrak{S}_9 : f_{213,9}(\pi) = f\}, \qquad f \in \mathbb{Z}_{\geq 0}.$$

Applying a second counter to `counts.values()` gives its positive bins. Because the output universe is all of $\mathfrak{S}_9$, the unattained outputs must also be counted:

$$h_9(0) = 9! - \#\{\pi : f_{213,9}(\pi) > 0\} = 362\,880 - 97\,278 = 265\,602. \tag{4}$$

This step is essential: zero-fertility outputs do not appear among the dictionary keys.

The archived result is transcribed in Table 1. The completed original program also checked the exact accounting identities

$$\sum_{f \geq 0} h_9(f) = 362\,880, \qquad \sum_{f \geq 0} f\, h_9(f) = 362\,880. \tag{5}$$

The first counts all outputs, including zero fibers; the second partitions all inputs by their output. These consistency checks do not replace the coverage argument, but detect missing mass in the recorded histogram. The asserted support was tested *after* the counts were computed; the expected set did not generate the counts.

The full census has 97 278 positive-fertility outputs. Its support is exactly the right-hand side of (2): there are 64 distinct positive fertility values, the largest is 128, and 55 is the unique missing value in $\{1, \ldots, 64\}$. These are statements about $n = 9$ only.

4 Complement conjugacy, including empty fibers

For a fixed length n, let c_n act entrywise by $x \mapsto n + 1 - x$. It is an involutive bijection of $\mathfrak{S}_n$. Complement symmetry is already stated in [1, Theorem 3.5 and Corollary 3.6]; we give the complete operation-level argument to fix the orientation and the zero case.

Lemma 2. *For every $n \geq 1$ and $\tau \in \mathfrak{S}_n$,*

$$\mathrm{SC}_{231}(c_n(\tau)) = c_n(\mathrm{SC}_{213}(\tau)). \tag{6}$$

Consequently $f_{231,n}(c_n(\pi)) = f_{213,n}(\pi)$ for every $\pi \in \mathfrak{S}_n$, including when either fiber is empty.

Proof. Run the two machines in parallel. Maintain the invariant that they have consumed the same number of inputs, their pending inputs are entrywise complements, and their stacks and outputs are entrywise complements in the same top-to-bottom and left-to-right orders. This holds initially with empty stacks and outputs.

For a triple of distinct entries, complementing values reverses all value comparisons. Its rank tuple (a, b, c) becomes $(4 - a, 4 - b, 4 - c)$; in particular $(2, 1, 3)$ becomes $(2, 3, 1)$. Therefore the prospective push is forbidden in the first machine exactly when it is forbidden in the second. If forbidden, both pop their current tops to their outputs and neither advances the pending input; the invariant is preserved. Otherwise both push their pending inputs and advance by one, again preserving the invariant. A stack of size below two admits the push on both sides. Once inputs are exhausted, their top-to-bottom drains also preserve the invariant. Termination then gives (6). This is value complement, with no reversal of positions or stack orientation.

For a fixed π, the map $\tau \mapsto c_n(\tau)$ sends the fiber of π under SC_{213} into the fiber of $c_n(\pi)$ under SC_{231}. Applying the same complement again gives the inverse, so the fibers are in bijection. The argument also proves that one fiber is empty if and only if the other is empty. Finally the output map $\pi \mapsto c_n(\pi)$ is bijective, so the entire fertility histograms, not just their supports, are equal. □

Proof of Theorem 1. The completed exhaustive computation described above gives Table 1 and hence A_9^{213} as in (2). Lemma 2 transfers this histogram to SC_{231} without a separate enumeration of that map. Since $55 \in \{1, \ldots, 2^{9-3}\}$ and $h_9(55) = 0$, the required fixed-length inclusion fails for each map. □

Table 1: Complete length-nine histogram for SC_{213}. Each pair of columns is a fertility f and the number $h_9(f)$ of outputs. The $f = 55$ row explicitly displays zero; every unlisted f also has count zero. The same histogram holds for SC_{231} by complement.

f	$h_9(f)$	f	$h_9(f)$	f	$h_9(f)$
0	265 602	22	164	44	7
1	40 163	23	58	45	2
2	11 353	24	98	46	5
3	11 534	25	75	47	5
4	9 915	26	182	48	10
5	5 736	27	69	49	4
6	3 595	28	79	50	1
7	3 684	29	37	51	1
8	2 039	30	55	52	4
9	1 297	31	91	53	3
10	1 308	32	60	54	4
11	1 470	33	7	55	0
12	854	34	3	56	5
13	569	35	3	57	9
14	663	36	12	58	3
15	667	37	5	59	2
16	607	38	7	60	4
17	164	39	5	61	1
18	144	40	3	62	3
19	182	41	2	63	6
20	148	42	9	64	9
21	69	43	4	128	1

5 Recorded execution and trust boundary

The original run was recorded on 13 October 2026. It started at 04:27:46.4455254 UTC and ended at 04:27:51.1014079 UTC. The process exit code is 0, the stop reason is null, and stderr is empty. The program's internal census timer records 4.4192823 seconds. The process monitor records an observed peak working set of 31 113 216 bytes, sampled every 50 ms; this is the archived measurement, not a new measurement or a claim about total system memory. The original receipt identifies an interpreter executable but does not record its version, so none is inferred here.

The original program first computed and saved the complete histogram, then compared it with an author-supplied histogram. The recorded stdout confirms that comparison passed. The result is supported by the actual independent enumeration and its exit record, not merely by agreement between two previously supplied data files. This note does not claim a new execution of either program.

The data file `data/n9.recomputed.json` is preserved byte-for-byte from that run. Its SHA-256 is

`ccf28642e4004512df0bd4054c0f98ce85d007442cc42effa17e697d2bbed882`

The distributed stdout and empty stderr are likewise unchanged; `evidence/EXECUTION.json` is a path-free summary binding the original script, receipt, monitor, result and output hashes.

The portable program `code/reproduce_n9.py` keeps the original stack function, input iteration, counters, assertions and result construction unchanged at the abstract-syntax-tree level. Its only adaptations are a neutral header and stdout-only output, replacing the old file-writing and path-specific comparison footer. This *packaged entry has not been executed*. Static preservation of the mathematics does not turn its new file bytes into an already executed artifact; the completed run remains bound to the original script. Further details and hashes are in the reproducibility note.

This is an exhaustive computational counterexample, conditional on the short implementation, Python's permutation and integer/container semantics, and the archived execution record. It is not a formal kernel certificate. The finite table and retained code permit direct scrutiny without claiming new independent human peer review.

6 What is and is not contradicted

Kemeklis's Theorem 1.1 [1, p. 2] has different quantifiers: for every positive integer f and every $\sigma \in \mathfrak{S}_3$, there exist a length n and a permutation $\pi \in \mathfrak{S}_n$ whose fertility is f. That length may depend on f and σ. Absence of fertility 55 at one fixed length therefore does not contradict it. No length-ten calculation or witness is needed for this distinction or for the refutation at length nine.

No minimal-counterexample-length claim is made. No simultaneous avoidance machine is considered. This note neither rechecks larger scans nor certifies worldwide publication priority. Prepared with AI assistance.

References

[1] Jurgis Kemeklis, *Fertility Numbers of Consecutive S_3 Pattern-Avoiding Stack-Sorting Maps*, arXiv:2408.05378v2, 15 September 2024. Algorithm p. 1; Theorem 1.1 p. 2; Theorem 3.5 and Corollary 3.6; Conjecture 4.1 p. 11.